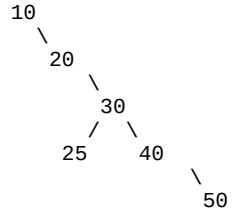
CASE STUDY – BST and AVL Trees in Library Book Management System

CO1: Apply Binary Search Trees (BST) and self-balancing AVL Trees to construct, insert, delete, and search hierarchical datasets, demonstrating correct rotation operations and O(log n) performance.

Scenario:
A university library named SmartLibrary stores book accession IDs using a Binary Search Tree (BST). Today's insertion order is: 10, 20, 30, 40, 50, 25. The library later upgrades the system to an AVL Tree for better performance.

(a) Construct the Plain BST

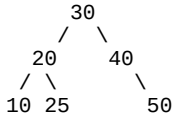
Insert the sequence: 10, 20, 30, 40, 50, 25 into a BST and find the height.



Height of BST = 4

(b) Construct AVL Tree and Show Rotations

Insert the same sequence into an AVL Tree and perform required rotations.



Height of AVL Tree = 2

Sub Question in (b): Java AVL Program

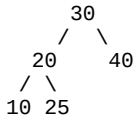
```
class AVLNode {
    int key, height;
    AVLNode left, right;

    AVLNode(int d) {
        key = d;
        height = 1;
    }
}

class AVLTree {

    AVLNode root;

    int height(AVLNode N) {
        if (N == null)
            return 0;
        return N.height;
    }

    int max(int a, int b) {
        return (a > b) ? a : b;
    }

    AVLNode rightRotate(AVLNode y) {

        AVLNode x = y.left;
```

```
    AVLNode T2 = x.right;

    x.right = y;
    y.left = T2;

    y.height = max(height(y.left), height(y.right)) + 1;
    x.height = max(height(x.left), height(x.right)) + 1;

    return x;
}

AVLNode leftRotate(AVLNode x) {

    AVLNode y = x.right;
    AVLNode T2 = y.left;

    y.left = x;
    x.right = T2;

    x.height = max(height(x.left), height(x.right)) + 1;
    y.height = max(height(y.left), height(y.right)) + 1;

    return y;
}

int getBalance(AVLNode N) {
    if (N == null)
        return 0;

    return height(N.left) - height(N.right);
}

AVLNode insert(AVLNode node, int key) {

    if (node == null)
        return new AVLNode(key);

    if (key < node.key)
        node.left = insert(node.left, key);

    else if (key > node.key)
        node.right = insert(node.right, key);

    else
        return node;

    node.height = 1 + max(height(node.left),
                          height(node.right));

    int balance = getBalance(node);

    if (balance > 1 && key < node.left.key)
        return rightRotate(node);

    if (balance < -1 && key > node.right.key)
        return leftRotate(node);

    if (balance > 1 && key > node.left.key) {
        node.left = leftRotate(node.left);
        return rightRotate(node);
    }

    if (balance < -1 && key < node.right.key) {
        node.right = rightRotate(node.right);
        return leftRotate(node);
    }

    return node;
}

void preOrder(AVLNode node) {
    if (node != null) {
        System.out.print(node.key + " ");
        preOrder(node.left);
        preOrder(node.right);
    }
}

public static void main(String[] args) {
    AVLTree tree = new AVLTree();
```

```
            tree.root = tree.insert(tree.root, 10);
            tree.root = tree.insert(tree.root, 20);
            tree.root = tree.insert(tree.root, 30);
            tree.root = tree.insert(tree.root, 40);
            tree.root = tree.insert(tree.root, 50);
            tree.root = tree.insert(tree.root, 25);

            System.out.println("Preorder Traversal of AVL Tree:");
            tree.preOrder(tree.root);
        }
    }
```

(c) Delete Node from AVL Tree

Delete node 50 from the AVL Tree.



Conclusion

BST may become skewed and inefficient, whereas AVL Trees remain balanced using rotations. AVL Trees provide O(log n) complexity for searching, insertion, and deletion.

Rotation Types Summary

Rotation Type	Condition
LL Rotation	Left-Left imbalance
RR Rotation	Right-Right imbalance
LR Rotation	Left-Right imbalance
RL Rotation	Right-Left imbalance